

Université du Québec à Rimouski

Département de mathématiques, d'informatique et de génie

8INF976 - Sujet spécial en intelligence artificielle

Efficacité et compression

Rapport de lecture

Réalisé par : Pier-Luc Dupéré

Professeur : Mehdi Adda, PhD

Table des matières

1 Quantification [1] 1

2 Élagage (pruning) [2] 3

3 Distillation [3] 5

4 LoRA/QLoRA [4] 7

5 Justification du choix méthodologique 10

Table des figures

1.1 Une technique de quantification qui consiste à réduire intentionnellement la précision des représentations numériques. 1

2.1 Élagage de SparseGPT en une seule étape. 3

4.1 LoRA réduit fortement le nombre de paramètres à optimiser et la mémoire consacrée aux gradients. 7

4.2 Le modèle QLoRA avec une précision de 4 bits qui effectue la pagination en mémoire GPU. 8

Liste des tableaux

4.1 Avantages et inconvénients des techniques de compression 9

Chapitre 1

Quantification [1]

La quantification est un processus qui consiste à représenter des données numériques avec une précision réduite afin d'en diminuer la taille ou d'en simplifier le traitement.Cette réduction de la précision permet de limiter les besoins en stockage, en bande passante ou en puissance de calcul, au prix d'une perte d'information généralement acceptable. Cela permet donc, l'optimisation des ressources et une meilleur accessibilité.

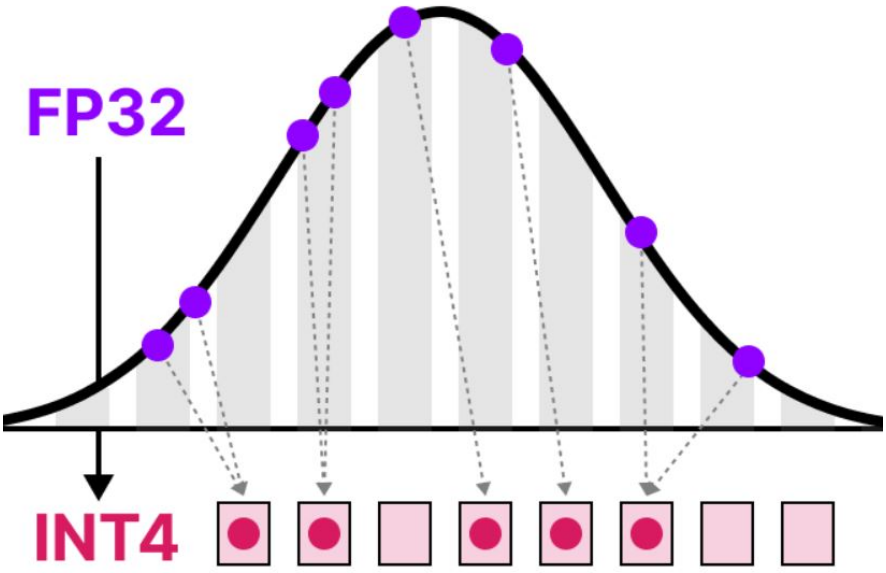


FIGURE 1.1 – Une technique de quantification qui consiste à réduire intentionnellement la précision des représentations numériques.

Il existe différentes stratégies existent pour quantifier les grands modèles de langage :

— **Quantification post-entraînement (PTQ) :** fait référence à l'ajustement des poids du modèle une fois l'entraînement terminé.
— **Quantification pendant l'entraînement (QAT) :** intègre le processus de quan-

tification pendant la formation du modèle. De plus, les modèles peuvent apprendre à s'adapter aux contraintes de précision dès le départ.

La quantification peut entraîner des pertes lorsque les LLM sont des modèles généralistes. Quantification pendant l'entraînement peut manqué d'efficacité en raison de son coût d'entraînement élevé. La quantification post-entraînement peut entraîner une légère dégradation des performances, particulièrement lorsque la précision utilisée est très faible.

AWQ (Activation Aware Weight Quantization) est une méthode de quantification qui atteint une excellente compression avec une perte de qualité minimale. L'idée derrière AWQ est que tous les poids d'un modèle ne sont pas également importants. AWQ identifie les poids les plus importants en examinant les motifs d'activation. La quantification traditionnelle traite tous les poids de la même manière. AWQ applique plus de précision aux poids ayant le plus grand impact sur la sortie. Le processus fonctionne en analysant un petit ensemble de données de calibration. AWQ mesure quels poids ont la plus forte influence sur les résultats.

La PTQ ne nécessite pas de ré-entraîner le modèle ni de modifier l'ensemble de ses paramètres. Elle consiste à quantifier un modèle déjà entraîné à l'aide d'un petit jeu de données de calibration.

Chapitre 2

Élagage (pruning) [2]

L'élagage est le processus consistant à éliminer les poids, neurones ou couches inutiles d'un réseau de neurones afin d'obtenir un modèle plus compact, plus rapide et plus efficace.

L'identification des redondances : La suppression de composants : Le réglage fin (fine-tuning) :

— **L'identification des redondances :** repérer les paramètres qui contribuent peu aux résultats du modèle.
— **La suppression de composants :** supprimer ces paramètres sans nuire de manière significative à la précision.
— **Le réglage des paramètres :** ré-entraîner le modèle élagué pour restaurer les performances perdues.

SparseGPT procède en ramenant le problème de l'élagage (ne seule étape) à un ensemble d'instances de régression creuse à très grande échelle. Il résout ensuite ces instances grâce à un nouvel algorithme de résolution approchée pour la régression creuse, suffisamment efficace pour s'exécuter en quelques heures sur les plus grands modèles GPT accessibles au public (175 milliards de paramètres) à l'aide d'un seul GPU

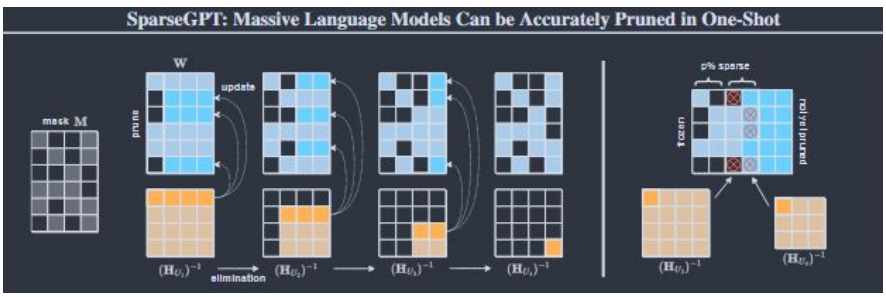


FIGURE 2.1 – Élagage de SparseGPT en une seule étape.

La compression après entraînement s'effectue généralement en décomposant le problème

de compression du modèle complet en sous-problèmes propres à chaque couche. Il existe plusieurs façons de faire de l'optimisation via de l'élagage :

— **Élagage non structuré :** Supprimer les poids ayant les plus faibles valeurs absolues
— **Élagage structuré :** Suppression de canaux, neurones, têtes d'attention ou blocs entiers
— **Élagage itératif ou dynamique :** Élaguer graduellement le modèle pendant l'entraînement, lui permettant de s'adapter en fonction de l'ampleur de la variation des poids.
— **Élagage global ou local :** Supprimer les plus petits poids sur l'ensemble du modèle global. Enlève des poids au sein de couches individuelles.

Chapitre 3

Distillation [3]

On peut utiliser la plusieurs façons pour transférer ses connaissances vers un modèle plus petit et plus facile à déployer. Le concept consiste à transférer les connaissances du modèle lourd vers un modèle plus compact, il est possible d'entraîner ce dernier à généraliser de la même façon que le grand modèle. On peut utiliser la moyenne arithmétique ou géométrique de leurs distributions prédictives individuelles comme cibles souples.

Une solution appelée la distillation, consiste à augmenter la température de la fonction finale jusqu'à ce que le modèle volumineux produise un ensemble de cibles suffisamment lissées. Plus T est élevé, plus la distribution de probabilité est douce et uniforme. À l'inverse, quand T se rapproche de 1.

La distillation consiste à transférer des connaissances vers le modèle distillé en l'entraînant sur un ensemble de transfert pour chaque cas de cet ensemble. On utilise une distribution cible lissée générée par le modèle volumineux avec une température élevée dans sa fonction de sa température.

La distillation des connaissances repose sur deux modèles :

— **Le modèle enseignant (teacher) :** est un modèle volumineux et performant, préalablement entraîné.
— **Le modèle distillé (student) :** est un modèle plus petit, entraîné pour reproduire les prédictions du modèle enseignant.

La température agit en fonction de la formule ci-dessous :

$$q_i^{(T)} = \text{softmax}\left(\frac{z_i^t}{T}\right) = \frac{\exp\left(z_i^t/T\right)}{\sum_{j=1}^{C} \exp\left(z_j^t/T\right)} \tag{3.1}$$

— z_i^t est le vecteur de logit du modèle enseignant pour la classe i ;

— C est le nombre total de classes ;
— T est la température ;
— $q_i^{(T)}$ est la probabilité produite par le modèle enseignant.

Le problème de cette formulation est qu'elle tend à produire des distributions très tranchées. Une classe domine largement et toutes les autres reçoivent des probabilités proches de zéro. L'information sur les similarités entre classes est ainsi perdue. Cela peut entraîner le sur apprentissage.

Dans cet article , les effets de la combinaison de modèles acoustiques basés sur des réseaux de neurones profonds (DNN) utilisés pour la reconnaissance automatique de la parole. La stratégie de distillation proposée permet de condenser un ensemble de modèles en un seul modèle.

Chapitre 4

LoRA/QLoRA [4]

LoRA (Low-rank Adapters) : Le réglage fin par adaptateurs de rang faible (LoRA) est une méthode qui réduit les besoins en mémoire en utilisant un petit ensemble de paramètres entraînables sans mettre à jour l'intégralité des paramètres du modèle. LoRA decompose la mise a jour en deux matrices de rang réduit, ce qui augmente l'efficacité de LoRA en termes de mémoire et de vitesse.

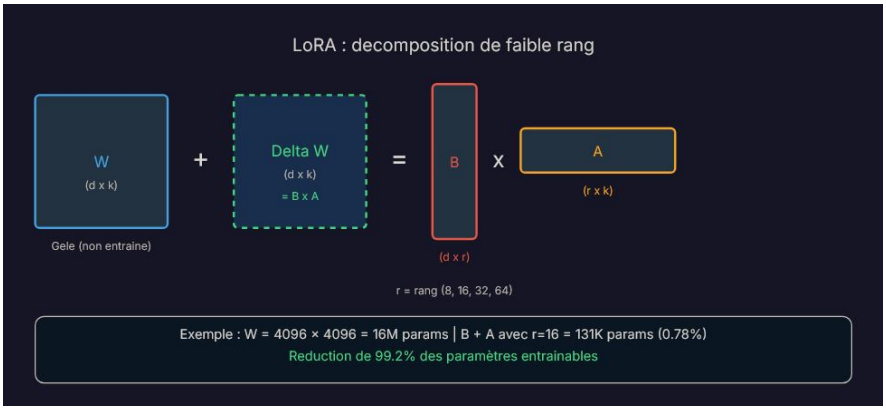


FIGURE 4.1 – LoRA réduit fortement le nombre de paramètres à optimiser et la mémoire consacrée aux gradients.

QLoRA est une méthode de réglage fin efficace en paramètres qui combine un modèle pré-entraîné figé et quantifié en 4 bits avec des adaptateurs LoRA de bas rang.

QLoRA permet notamment d'ajuster des modèles très volumineux, jusqu'à 65 milliards de paramètres, sur un seul GPU de 48 Go, tout en conservant des performances proches d'un réglage fin complet en précision 16 bits. Le banc d'essai Vicuna, atteignant 99,3 % des performances de ChatGPT tout en ne nécessitant que 24 heures d'ajustement fin sur un seul GPU.

— **Le format 4-bit NormalFloat NF4 :** un nouveau type de données optimal au sens de la théorie de l'information pour des poids suivant une distribution normale.

Cette méthode repose sur la quantification par quantiles, une méthode optimale du point de vue de la théorie de l'information qui garantit que chaque intervalle de quantification reçoit un nombre égal de valeurs issues du tenseur d'entrée.

— **Double quantification :** la double quantification, pour réduire l'empreinte mémoire moyenne en quantifiant les constantes de quantification. La première étape comme des entrées pour une seconde quantification. Cette étape produit des constantes de quantification quantifiées. La seconde étapes, on utilise des nombres à virgule flottante sur 8 bits avec une taille de bloc de 256, car aucune dégradation des performances n'est observée avec une quantification sur 8 bits.

— **Optimiseurs paginés :** pour gérer les pics de consommation mémoire et evite tout dépassent. On exploitent la fonctionnalité « Unified Memory 3 » de NVIDIA, qui assure des transferts automatiques de pages entre le CPU et le GPU. Cela permet un traitement GPU sans erreur même lorsque la mémoire du GPU vient à manquer ponctuellement.

QLORA utilise un type de données de stockage à faible précision (généralement 4 bits) et un type de données de calcul (généralement BFloat16). En pratique, cela signifie que chaque fois qu'un tenseur de poids QLORA est utilisé, nous déquantifions le tenseur en BFloat16, puis nous effectuons une multiplication matricielle en 16 bits.

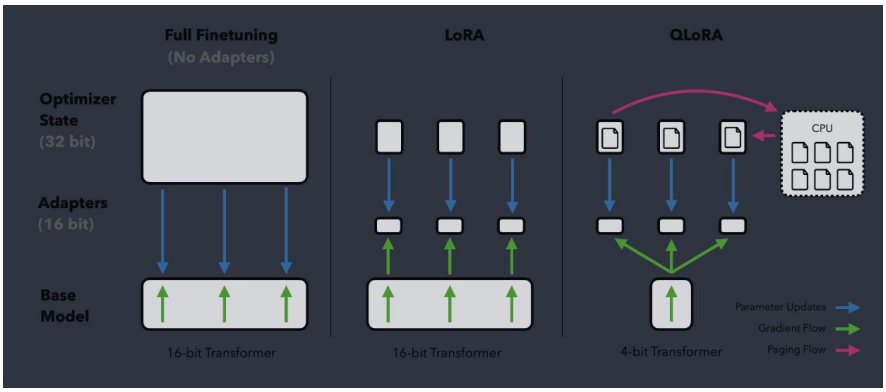


FIGURE 4.2 – Le modèle QLoRA avec une précision de 4 bits qui effectue la pagination en mémoire GPU.

TABLE 4.1 – Avantages et inconvénients des techniques de compression

Technique	Avantages	Inconvénients
Quantification	— Réduit la taille du modèle et sa consommation mémoire. — La quantification post-entraînement est relativement rapide à mettre en oeuvre. — Des méthodes comme AWQ préservent mieux les poids importants en analysant les activations.	— La réduction de la précision peut entraîner une perte de performance. — La quantification peut dégrader les résultats des modèles généralistes. — La quantification consciente de l'entraînement augmente le coût et la durée de formation.
Élagage	— Supprime les poids, neurones ou couches peu importants. — Réduit le nombre de paramètres et les besoins en mémoire. — L'élagage structuré peut produire des gains réels de vitesse.	— La suppression de paramètres peut entraîner une perte de précision. — L'élagage non structuré ne garantit pas toujours une accélération réelle. — L'identification des paramètres redondants peut être complexe.
Distillation	— Transfère les connaissances d'un modèle volumineux vers un modèle plus compact. — Réduit les besoins en mémoire et en puissance de calcul. — Les cibles souples fournissent des informations sur les similarités entre les classes.	— Le modèle étudiant peut être moins précis que le modèle enseignant. — Les erreurs et les biais du modèle enseignant peuvent être transférés. — Le choix de la température et de la fonction de perte influence fortement les résultats.
LoRA / QLoRA	— Le format NF4 (4 bits) améliore la représentation des poids quantifiés. — La double quantification réduit davantage l'empreinte mémoire. — Les optimiseurs paginés limitent les problèmes liés aux pics de consommation mémoire.	— Le modèle de base quantifié doit toujours être chargé pendant l'inférence. — Le choix du rang des matrices influence la qualité et le coût. — La configuration est plus complexe que celle de LoRA seul.

Chapitre 5

Justification du choix méthodologique

Pour ce TP2, j'ai choisi d'essayer comme techniques de compression de modèles, la quantification et l'élagage (pruning).

La quantification est effectuée au chargement du modèle avec BitsAndBytesConfig et la bibliothèque bitsandbytes. Pour étudier l'élagage, j'applique un élagage non structuré L1 aux poids des couches linéaires avec la librairie pytorch.

Mon GPU n'étant pas très puissant, j'ai choisi de ne pas implémenter QLoRA lors de mon travail pratique. De plus, je n'ai pas retenu la technique de distillation pour mon travail pratique. Je crois qu'elle est plus complexe à mettre en oeuvre et nécessite un temps d'entraînement plus long.

Bibliographie

[1] Lin, J., Tang, J., Tang, H., Yang, S., Dang, X., Gan, C. et Han, S., *AWQ : activation-aware weight quantization for on-device llm compression and acceleration*, https://arxiv.org/pdf/2306.00978v6, consulté le 30 septembre 2026.

[2] Frantar, E. et Alistarh, D., *SparseGPT : Massive Language Models Can be Accurately Pruned in One-Shot*, https://arxiv.org/pdf/2301.00774v3, consulté le 30 septembre 2026.

[3] Hinton, G., Vinyals, O. et Dean, J., *Distilling the Knowledge in a Neural Network*, https://arxiv.org/pdf/1503.02531v1, consulté le 30 septembre 2026.

[4] Dettmers, T., Pagnoni, A., Holtzman, A. et Zettlemoyer, L., *QLORA : Efficient Finetuning of Quantized LLMs*, https://arxiv.org/pdf/2305.14314v1, consulté le 30 septembre 2026.